

Data Engineering & Inteligencia Artificial

Detección de anomalías mediante aprendizaje competitivo

Implementación de una red neuronal competitiva
para el análisis de datos financieros y detección de anomalías

Proyecto 1

Autora: Argiñe Delgado

Abril 2026

Índice

1. Resumen	3
2. Introducción	4
3. Objetivos	4
3.1. Objetivo general	4
3.2. Objetivos específicos	4
4. Descripción del dataset	5
5. Preprocesamiento de los datos	5
5.1. Carga del dataset	5
5.2. Normalización	6
6. Fundamentos del aprendizaje competitivo	6
7. Arquitectura de la red	7
8. Inicialización de las neuronas	7
9. Proceso de entrenamiento	7
10. Cálculo de la neurona ganadora (BMU)	8
11. Actualización de los pesos	8
12. Visualización y análisis de resultados	9
12.1. Interpretación de los pesos	9
13. Asignación de perfiles de riesgo	9
14. Detección de anomalías	10
15. Evaluación de una nueva transacción	10
16. Asignación de observaciones a neuronas	11
17. Cálculo del riesgo promedio	11
18. Gráfico de riesgo por neurona	11

19.Limitaciones del modelo	12
19.1. Número de neuronas	12
19.2. Umbral de anomalía	12
19.3. Etiquetas de los perfiles	12
20.Posibles mejoras	13
21.Conclusiones	14

1. Resumen

En este proyecto se desarrolla una aplicación de aprendizaje automático basada en aprendizaje competitivo. El objetivo principal es estudiar la distribución de diferentes observaciones de un conjunto de datos relacionado con operaciones financieras y utilizar una red neuronal competitiva para identificar patrones y posibles anomalías.

Antes del entrenamiento, los datos se normalizan mediante la herramienta **StandardScaler**. Posteriormente se inicializan cinco neuronas con pesos obtenidos a partir de observaciones aleatorias del conjunto de datos.

El algoritmo utiliza una estrategia de aprendizaje competitivo. Para cada observación se calcula la distancia respecto a todas las neuronas, y la neurona que se encuentra más cerca es seleccionada como neurona ganadora o BMU (*Best Matching Unit*). Únicamente esta neurona actualiza sus pesos, desplazándose hacia la observación presentada.

Finalmente, los pesos obtenidos se utilizan para calcular un indicador de anomalía, que se basa en la distancia mínima entre una nueva observación y cualquiera de los prototipos aprendidos.

Los resultados permiten estudiar cómo las neuronas representan diferentes regiones del espacio de datos y cómo puede utilizarse la distancia respecto a estos prototipos para detectar observaciones potencialmente anómalas.

2. Introducción

El análisis de datos financieros constituye una de las aplicaciones importantes dentro del campo de la inteligencia artificial y el aprendizaje automático. Las entidades financieras trabajan diariamente con grandes cantidades de transacciones, por lo que resulta necesario disponer de herramientas capaces de identificar patrones de comportamiento y detectar operaciones que se alejen de dichos patrones.

Una de las estrategias utilizadas para este tipo de problemas consiste en el aprendizaje no supervisado. A diferencia del aprendizaje supervisado, en este caso no es necesario disponer de una etiqueta que indique previamente si cada observación es normal o anómala.

El aprendizaje competitivo es una técnica de aprendizaje no supervisado en la que diferentes neuronas compiten para representar cada observación. La neurona que se encuentra más cerca de una determinada observación se convierte en la neurona ganadora.

Durante el entrenamiento, la neurona ganadora modifica sus pesos para acercarse a la observación. Después de repetir este proceso numerosas veces, las neuronas terminan representando diferentes regiones del espacio de datos.

En este proyecto se implementa este procedimiento utilizando Python, apoyándose en librerías como NumPy, Pandas, Matplotlib, Seaborn y Scikit-learn.

La finalidad no es construir un sistema bancario de producción, sino comprender el funcionamiento de una red neuronal competitiva y estudiar cómo puede emplearse la distancia a los prototipos aprendidos como indicador de anomalía.

3. Objetivos

3.1. Objetivo general

Desarrollar e implementar una red neuronal basada en aprendizaje competitivo capaz de identificar patrones dentro de un conjunto de datos financieros y utilizar los prototipos aprendidos para detectar posibles observaciones anómalas.

3.2. Objetivos específicos

Los objetivos específicos del proyecto son:

- Cargar el conjunto de datos mediante Pandas.
- Seleccionar las variables V1 y V2 para realizar un análisis bidimensional.
- Normalizar los datos utilizando `StandardScaler`.

- Inicializar cinco neuronas competitivas.
- Implementar un algoritmo de aprendizaje competitivo desde cero.
- Calcular la distancia entre las observaciones y las neuronas.
- Identificar la neurona ganadora o BMU.
- Actualizar los pesos de la neurona ganadora.
- Visualizar los pesos iniciales y finales.
- Implementar un score de anomalía.
- Clasificar las observaciones según la neurona más cercana.
- Calcular el riesgo promedio asociado a cada neurona.
- Analizar las limitaciones del método y proponer mejoras.

4. Descripción del dataset

El proyecto utiliza un archivo denominado `CreditcardDataset.csv`. El archivo contiene diferentes variables relacionadas con operaciones financieras.

En este proyecto se seleccionan únicamente dos de esas variables, las denominadas `V1` y `V2`, extrayéndolas del dataframe original como una matriz de valores numéricos que servirá de entrada al modelo.

La utilización de dos variables tiene principalmente una finalidad didáctica y visual. Al trabajar con dos dimensiones es posible representar gráficamente las observaciones y observar el desplazamiento de los prototipos durante el entrenamiento.

Cada observación del conjunto de datos queda representada, por tanto, como un punto en un plano de dos coordenadas: la primera correspondiente a la variable `V1` y la segunda a la variable `V2`.

5. Preprocesamiento de los datos

Antes de comenzar el entrenamiento es necesario preparar los datos.

5.1. Carga del dataset

La carga se realiza utilizando Pandas, leyendo el archivo `CreditcardDataset.csv` desde la ruta local donde se encuentra almacenado y guardando su contenido en un dataframe.

Posteriormente se extraen las dos variables seleccionadas, V1 y V2, y se convierten en una matriz de dos columnas que constituirá la entrada del modelo.

5.2. Normalización

El siguiente paso consiste en estandarizar los datos mediante la herramienta **StandardScaler** de Scikit-learn. Este procedimiento transforma cada variable de modo que su media pase a ser cero y su desviación estándar pase a ser uno, es decir, resta a cada valor la media de su variable y el resultado lo divide entre la desviación estándar de esa misma variable.

Esta transformación es importante porque el algoritmo utiliza la distancia entre puntos para decidir qué neurona representa mejor cada observación. Si las variables tuvieran escalas muy diferentes, la variable con mayor rango de valores dominaría el cálculo de la distancia y distorsionaría el resultado del entrenamiento. Al normalizar ambas variables se garantiza que las dos contribuyan de forma equilibrada.

6. Fundamentos del aprendizaje competitivo

El aprendizaje competitivo es un tipo de aprendizaje no supervisado en el que las neuronas compiten para determinar cuál representa mejor cada observación.

A diferencia de una red neuronal tradicional, no todas las neuronas se actualizan al mismo tiempo. Para cada observación existe una única neurona ganadora.

El funcionamiento general puede resumirse de la siguiente manera:

1. Se presenta una observación a la red.
2. Se calcula la distancia entre la observación y cada neurona.
3. Se selecciona la neurona más cercana.
4. La neurona ganadora actualiza sus pesos.
5. El proceso se repite para todas las observaciones.
6. El procedimiento completo se repite durante varias épocas.

Con el paso de las iteraciones, las neuronas se especializan en diferentes regiones del espacio de datos, de manera que cada una termina representando un grupo de observaciones con características similares.

7. Arquitectura de la red

La red utilizada en este proyecto está formada por cinco neuronas competitivas. El número de neuronas se fija de forma manual al comienzo del programa, al igual que la dimensión de entrada, que en este caso coincide con el número de variables seleccionadas del dataset, es decir, dos.

Como se utilizan dos variables, cada neurona posee dos pesos, uno asociado a la variable V1 y otro asociado a la variable V2. El conjunto completo de las cinco neuronas puede entenderse como una tabla de cinco filas y dos columnas, donde cada fila representa los dos pesos de una neurona concreta.

8. Inicialización de las neuronas

Los pesos iniciales se obtienen seleccionando cinco observaciones aleatorias del conjunto de datos ya normalizado. Es decir, en lugar de partir de valores arbitrarios, cada neurona comienza situada exactamente en la posición de una transacción real del dataset, elegida al azar y sin repetición entre las cinco.

Para poder repetir el experimento y obtener siempre los mismos resultados, se fija de antemano una semilla aleatoria. De este modo, cada vez que se ejecuta el programa se seleccionan las mismas cinco observaciones iniciales, lo que permite comparar resultados entre distintas ejecuciones.

Una vez elegidas estas cinco observaciones, se guarda también una copia de esos pesos iniciales, ya que más adelante se utilizarán para compararlos visualmente con los pesos obtenidos tras el entrenamiento.

La red comienza, por tanto, con cinco prototipos situados en diferentes posiciones del espacio de datos.

9. Proceso de entrenamiento

El entrenamiento se realiza durante diez épocas, entendiendo por época una pasada completa por todas las observaciones del conjunto de datos.

La tasa de aprendizaje utilizada, que determina cuánto se desplaza la neurona ganadora en cada actualización, no permanece constante durante todo el proceso, sino que disminuye progresivamente a medida que avanzan las épocas. Al comienzo del entrenamiento se parte de un valor inicial, y en cada nueva época ese valor se reduce de forma proporcional al número de épocas que ya se han completado, de manera que en la última época la tasa de aprendizaje es prácticamente nula.

Esta disminución progresiva tiene una razón práctica: al principio del entrenamiento conviene que los prototipos se muevan con libertad y rapidez para alejarse de su

posición inicial y aproximarse a las zonas donde realmente se concentran los datos. Sin embargo, a medida que avanza el entrenamiento interesa que los ajustes sean cada vez más pequeños y precisos, para que los prototipos se estabilicen en lugar de oscilar continuamente.

Dentro de cada época, además, el orden en el que se presentan las observaciones a la red se mezcla de forma aleatoria, de modo que el modelo no aprenda ningún patrón relacionado con el orden original de las filas del dataset.

10. Cálculo de la neurona ganadora (BMU)

Para cada observación que se presenta a la red, el primer paso consiste en calcular su distancia respecto a las cinco neuronas. Esta distancia se mide de la forma habitual en el espacio bidimensional: se compara la posición de la observación con la posición de cada neurona, teniendo en cuenta tanto la diferencia en la primera coordenada como la diferencia en la segunda, y combinando ambas diferencias para obtener un único valor que representa lo alejadas o cercanas que están la observación y esa neurona.

Una vez calculadas las cinco distancias, correspondientes a las cinco neuronas, se identifica cuál de ellas es la más pequeña. La neurona asociada a esa distancia mínima es la que se declara neurona ganadora o BMU para esa observación concreta. Esta neurona es, por tanto, la que en ese momento representa mejor a la observación presentada.

11. Actualización de los pesos

Una vez identificada la neurona ganadora, únicamente sus pesos se actualizan; el resto de neuronas permanece sin cambios en esa iteración.

La actualización consiste en desplazar ligeramente la posición de la neurona ganadora en dirección a la observación que se acaba de presentar. Para ello se calcula primero la diferencia entre la posición de la observación y la posición actual de la neurona ganadora; esta diferencia indica hacia qué dirección y a qué distancia debería moverse la neurona para coincidir exactamente con la observación.

A continuación, esa diferencia se multiplica por la tasa de aprendizaje correspondiente a la época actual, y el resultado se suma a la posición actual de la neurona. De esta manera, la neurona no salta directamente hasta la posición exacta de la observación, sino que se desplaza solo una fracción de esa distancia, determinada por el valor de la tasa de aprendizaje en ese momento del entrenamiento.

Repitiendo este proceso observación tras observación, y época tras época, cada neurona va desplazándose progresivamente hacia la zona del espacio de datos donde se concentran las observaciones que con más frecuencia la eligen como ganadora.

12. Visualización y análisis de resultados

Para estudiar visualmente el comportamiento del modelo se utilizan las librerías Matplotlib y Seaborn.

En primer lugar se representa la densidad de los datos normalizados mediante un mapa de densidad que colorea con distintas tonalidades las zonas del plano donde se concentran más observaciones, permitiendo identificar de un vistazo las regiones más pobladas del espacio de datos.

Sobre ese mapa de densidad se superponen, en primer lugar, los pesos iniciales de las cinco neuronas, marcados con un símbolo distintivo, y en segundo lugar los pesos finales obtenidos tras el entrenamiento, marcados con un símbolo diferente. Además, se añaden flechas que conectan la posición inicial de cada neurona con su posición final, de modo que se aprecia claramente el recorrido que ha realizado cada prototipo durante el aprendizaje.

La comparación entre ambos conjuntos de pesos permite observar cómo los prototipos cambian de posición durante el entrenamiento, desplazándose desde puntos elegidos al azar hacia zonas donde realmente existe una concentración importante de observaciones.

12.1. Interpretación de los pesos

Los pesos iniciales representan simplemente la posición de partida de las neuronas, correspondiente a cinco observaciones elegidas al azar.

Los pesos finales, en cambio, representan los prototipos obtenidos después del proceso de aprendizaje, es decir, las posiciones que mejor resumen las distintas regiones del espacio de datos.

Si una neurona termina localizada en una región con una elevada concentración de observaciones, esto significa que ha aprendido a representar adecuadamente esa región del espacio, actuando como un punto de referencia para todas las observaciones cercanas a ella.

13. Asignación de perfiles de riesgo

Con fines exclusivamente ilustrativos, el programa asocia a cada una de las cinco neuronas una etiqueta descriptiva, como por ejemplo “Riesgo Bajo”, “Riesgo Medio”, “Fraude”, “Empresa” o “Particular”.

Estas etiquetas permiten visualizar los resultados de una manera más sencilla e intuitiva para el lector del informe.

Sin embargo, es importante destacar que estas categorías no son aprendidas directamente por el algoritmo. La red únicamente aprende posiciones o prototipos dentro del espacio de características definido por las variables V1 y V2, sin ningún conocimiento sobre lo que esas posiciones significan en términos de negocio.

Por tanto, afirmar que una neurona representa concretamente “Fraude” requeriría validar dicha interpretación con información real y etiquetas conocidas. Esta consideración es especialmente importante si el modelo se utilizara en una aplicación financiera real, ya que las etiquetas asignadas en este proyecto tienen un propósito puramente didáctico.

14. Detección de anomalías

Una de las aplicaciones principales de los prototipos aprendidos consiste en identificar observaciones alejadas de las regiones conocidas.

Para ello se define un indicador, denominado score de anomalía, que se calcula para cualquier observación nueva de la siguiente manera: se mide la distancia entre esa observación y cada una de las cinco neuronas entrenadas, y de esas cinco distancias se toma la más pequeña. Este valor mínimo constituye el score de anomalía de la observación.

La interpretación de este indicador es sencilla: si el prototipo más cercano a la nueva observación está muy próximo a ella, el score será pequeño, lo que indica que la observación se parece a patrones ya conocidos por la red. Si, por el contrario, la observación se encuentra lejos de los cinco prototipos, el score será elevado, lo que sugiere que se trata de una observación atípica o potencialmente anómala.

15. Evaluación de una nueva transacción

Para probar el modelo se introduce una nueva observación de ejemplo, definida por un par de valores correspondientes a las variables V1 y V2 de una transacción hipotética.

Antes de calcular el score, la nueva observación debe normalizarse utilizando exactamente el mismo objeto de escalado que se empleó durante el entrenamiento del modelo. Esto es fundamental, ya que no se debe crear un nuevo escalador para la nueva observación, pues eso haría que se perdiera la correspondencia con la escala utilizada durante el entrenamiento y los resultados dejarían de ser comparables con los prototipos aprendidos.

Una vez normalizada la nueva transacción, se calcula su score de anomalía frente a las cinco neuronas entrenadas, siguiendo el mismo procedimiento descrito en la sección anterior.

El programa establece un criterio sencillo: si el score obtenido supera un determinado umbral, la transacción se marca como una posible anomalía. Es importante señalar que este umbral se ha definido de forma manual en este proyecto, y que en una aplicación real debería determinarse mediante un análisis estadístico riguroso y el uso de datos históricos, en lugar de fijarse de manera arbitraria.

16. Asignación de observaciones a neuronas

Después del entrenamiento, el programa recorre todas las observaciones del conjunto de datos y, para cada una de ellas, calcula de nuevo cuál es la neurona más cercana, es decir, su neurona ganadora, siguiendo el mismo procedimiento que se utilizó durante el entrenamiento.

De esta manera, cada observación del dataset queda asociada a uno de los cinco grupos definidos por las neuronas, de forma similar a como funcionaría un algoritmo de agrupamiento. Esto permite analizar el comportamiento de cada neurona por separado, estudiando qué características tienen en común las observaciones que comparten la misma neurona ganadora.

17. Cálculo del riesgo promedio

El programa calcula, para cada una de las cinco neuronas, el riesgo promedio de las observaciones que tiene asignadas, utilizando para ello la segunda variable del dataset, correspondiente a V2.

Para llevar a cabo este cálculo se recorren todas las observaciones y, para cada una de ellas, se identifica primero a qué neurona fue asignada. A continuación, se acumula el valor de la variable V2 de esa observación en un contador asociado a esa neurona, y al mismo tiempo se incrementa en uno el número total de observaciones que dicha neurona ha recibido.

Una vez recorridas todas las observaciones, el riesgo acumulado de cada neurona se divide entre el número de observaciones que le han sido asignadas, obteniéndose así el riesgo promedio correspondiente a cada una de las cinco neuronas. Se añade además una pequeña cantidad al denominador para evitar problemas en el caso poco probable de que alguna neurona no tenga ninguna observación asignada.

18. Gráfico de riesgo por neurona

El último gráfico del programa representa el riesgo promedio de cada neurona mediante un gráfico de barras, en el que cada barra corresponde a una de las cinco neuronas y su altura refleja el valor de riesgo promedio calculado en la sección anterior.

El color de cada barra varía en función de su valor de riesgo, de manera que resulte sencillo distinguir visualmente qué neuronas presentan un riesgo más elevado y cuáles presentan un riesgo más bajo.

Además, el eje horizontal del gráfico muestra el perfil descriptivo asociado a cada neurona junto con el número de observaciones que tiene asignadas, lo que facilita interpretar de un vistazo tanto el tamaño como el nivel de riesgo de cada grupo.

Este gráfico permite responder preguntas como las siguientes:

- ¿Qué neurona concentra más observaciones?
- ¿Qué neurona presenta un mayor riesgo promedio?
- ¿Existen neuronas con muy pocas observaciones?
- ¿Existen grupos claramente diferenciados?

19. Limitaciones del modelo

Aunque el modelo permite demostrar el funcionamiento del aprendizaje competitivo, existen varias limitaciones que conviene tener en cuenta.

19.1. Número de neuronas

El número de neuronas se fija manualmente en cinco desde el principio del programa. Este valor no necesariamente es el óptimo para representar la estructura real de los datos. Sería interesante comparar diferentes cantidades de neuronas y estudiar cómo afecta al resultado del agrupamiento.

19.2. Umbral de anomalía

El umbral utilizado para decidir si una observación es anómala se establece de forma manual, sin ningún respaldo estadístico. En un sistema real, este umbral debería calcularse a partir de datos históricos, distribuciones estadísticas de las distancias observadas, o métricas de evaluación específicas para el problema de detección de fraude.

19.3. Etiquetas de los perfiles

Las etiquetas descriptivas asignadas a cada neurona, como “Fraude”, “Riesgo Bajo” o “Empresa”, no son aprendidas por el algoritmo, sino etiquetas interpretativas asignadas posteriormente por quien analiza los resultados. Para validar estas categorías sería necesario disponer de información adicional, como etiquetas reales de fraude confirmadas por la entidad financiera.

20. Posibles mejoras

El proyecto puede ampliarse mediante diferentes estrategias:

- Utilizar todas las variables disponibles en el dataset, en lugar de limitarse a dos.
- Aumentar el número de épocas de entrenamiento.
- Probar diferentes tasas de aprendizaje iniciales.
- Comparar diferentes cantidades de neuronas.
- Determinar de forma automática el umbral de anomalía, en lugar de fijarlo manualmente.
- Utilizar datos históricos de fraude para validar los resultados obtenidos.
- Comparar el algoritmo con K-Means.
- Comparar el modelo con Isolation Forest.
- Utilizar mapas autoorganizados (Self-Organizing Maps).
- Utilizar autoencoders para la detección de anomalías.
- Evaluar el modelo utilizando precisión, exhaustividad y F1-score, en caso de disponer de etiquetas reales.

21. Conclusiones

En este proyecto se ha desarrollado una red neuronal basada en aprendizaje competitivo con el objetivo de estudiar patrones presentes en datos financieros y detectar posibles observaciones anómalas.

El primer paso del proceso ha sido la carga y selección de las variables V1 y V2. Posteriormente se han normalizado mediante `StandardScaler`, lo que permite realizar el cálculo de distancias de forma más equilibrada entre ambas variables.

La red está formada por cinco neuronas. Cada neurona contiene un par de pesos que actúa como prototipo de una región determinada del espacio de datos.

Durante el entrenamiento, cada observación compite entre las cinco neuronas. La neurona más cercana es seleccionada como BMU y únicamente ella actualiza sus pesos, acercándose progresivamente a la observación presentada.

Como consecuencia del entrenamiento, los pesos finales representan diferentes regiones del conjunto de datos, cada una asociada a un grupo de observaciones con características similares.

Una de las partes más importantes del proyecto es el cálculo del score de anomalía, que mide la distancia de una nueva observación respecto al prototipo más cercano. Si esa distancia es elevada, la observación se encuentra alejada de los patrones aprendidos y puede considerarse potencialmente anómala.

Sin embargo, el modelo presentado debe considerarse una implementación educativa y experimental. Para utilizarlo en un sistema real de detección de fraude sería necesario disponer de más variables, validar los umbrales de forma estadística, utilizar datos etiquetados y realizar una evaluación rigurosa del rendimiento.

En conclusión, el proyecto permite comprender de forma práctica cómo un algoritmo de aprendizaje no supervisado puede utilizarse para descubrir estructuras dentro de los datos y construir un mecanismo sencillo de detección de anomalías.